

A Distributional Audit of a Public Julia Comparison Artifact for Weighted Reservoir Sampling with Replacement

Submission manuscript, version 11 — 5 October 2026.

Mohammad Haribat — Independent Researcher, Kuwait. Correspondence: mharibat@mew.gov.kw.

Abstract

A correct sampling construction does not ensure that its implementation follows the target distribution. We examine a public Julia comparison artifact for weighted reservoir sampling with replacement using an independent implementation, exact enumeration and native execution. The audit identifies defects in distinct-item tracking during conversion and in a binomial-count recurrence. The conversion failure path requires at least four outputs: with four equally weighted items and four draws, the original routine assigns probability $10/64$ to two pairs instead of $9/64$. A conditional-rank argument justifies the conversion repair, supported by 19 exact weighted checks. The corrected count shortcut matches its analytical law in 296 rational parameter combinations. Across twelve configurations, 60,000 fresh reservoirs quantify conversion effects at four sample fractions and three weight profiles. Paired normalized-mean differences are detected at 10% and 20% fractions for increasing and decreasing weights, with opposite signs. At 20%, the differences are 0.000968 and -0.000984 ; simultaneous family intervals exclude zero. Uniform-weight mean contrasts do not reject despite the distributional counterexample. Reanalysis of 2,880 timing records retains the observed SKIP ranking but leaves speedup magnitude uncertain. Selected outputs reproduce byte-for-byte after dependency installation on the same host. The results establish two defects in the comparison artifact and quantify their measured consequences. They neither certify general floating-point correctness nor reconstruct the historical environment used for the published experiments.

Introduction

For positive weights $w_1, \dots, w_N$, an unordered with-replacement sample of size m must have count vector

$$\mathbf{C} \sim \text{Multinomial}(m; w_1/W, \dots, w_N/W), \quad W = \sum_i w_i.$$

A sample can be sorted without changing this count law. Consequently, ordered-slot independence is not an appropriate implementation test when an interface deliberately returns ordered samples. Count vectors and occupancy partitions retain the relevant information under sorting.

Meligrana and Fazzone [1], [2] present a skip-based weighted reservoir method and comparisons with other methods. Our initial implementation, developed from the mathematical description before inspecting author code, was consistent with the tested sampling laws. Subsequent inspection of the linked comparison artifact exposed differences in initialization, count generation, conversion, and timing conventions. The research question is whether this public artifact implements its required count distributions, whether localized corrections remove observed discrepancies, and how the corrected methods compare under matched native timing boundaries.

The independent replication and source audit provide:

- Two exact implementation counterexamples, confirmed by native execution.
- Local repairs supported by a conditional-rank argument and a count-recurrence proof.
- Fresh-reservoir experiments isolating estimator effects across sample fractions and weight directions.
- Native timing comparisons and selected dependency-isolated replays, with explicit limits on historical reproducibility.

Related work

Weighted-key reservoir methods [3] produce ordered samples without replacement. The multinomial construction of Shekelyan et al. [4] uses that order to provide the next previously unseen item while allowing repeats of emitted items. The public artifact associated with [2] implements a conversion using a dynamic weighted sampler. Correctness requires consistent tracking of which distinct items have actually been emitted.

The algorithmic proof leaves two implementation questions unresolved: whether the shortcut computes the required recurrence and whether the converter updates its state consistently. We examine those questions in one pinned comparison file. Exact distributional testing itself is established methodology.

Method family / source	Maintained object	Retrieval requirement	Specific audit boundary
Weighted reservoir keys [3]	Weighted sample without replacement	Extract selected items	Does not itself provide a with-replacement count law
Online multinomial construction [4]	Ordered weighted reservoir plus repeat/new-item decisions	Convert selected ranks to repeated draws	Newly emitted ranks must advance the distinct-item state
Direct BIN comparison [2], [5]	Sample with replacement	Stored sample reference	Count shortcut must implement the binomial recurrence
SKIP [1], [2], [5]	Sample with replacement with skipped update events	Stored sample reference	Positive-truncated counts and initialization must satisfy the intended law
Present audit	Independent implementation, then pinned-source diagnostics	Separate borrowed and materialized interfaces	Establishes concrete counterexamples and measured repair effects; no new sampling method

Statistical testing of samplers and formal construction of correct samplers address different guarantees. Bagnall et al. [6] compile discrete probabilistic programs to samplers through a Coq-verified pipeline. The Julia routines examined here have no corresponding machine-checked guarantee. Bhattacharyya et al. [7] estimate distribution distance using subcube-conditional access for self-reducible samplers; that access model is not supplied by the reservoir interface studied here. Zeyen et al. [8] combine uniformity diagnostics and examine input-dependent outcomes for Boolean-formula samplers. Those diagnostics concern a different target distribution and interface. Our tests instead use the weighted product and count laws, with source-level interventions to identify the cause of a discrepancy. The accessible preprint of [8] was read alongside verified journal metadata; final-version wording was not independently matched.

Meligrana's software paper [9] describes StreamSampling.jl across weighted and unweighted sampling, with and without replacement, through package-level interfaces. Its package-level results cannot determine whether a released version shares the defects in this separate comparison file. We consulted its versioned preprint, not a matched final publisher text.

Kalibera and Jones [10] distinguish variation between builds, executions and iterations when designing performance experiments. Our saved timing campaign does not estimate all those levels. Our conclusions therefore concern the recorded paired measurements; variation across builds and independent executions remains unestimated.

Methods

The independent C++ implementation was frozen before author-code inspection. Native diagnostics then used the public comparison file [5] at commit 408dd06f8ea9e60b412e27a73f0f277bb590cb84, Julia 1.12.1, and pinned dependencies. The file is byte-identical at the subsequently retrieved default HEAD; the full provenance is in Supplement S1. Historical dependency equality and identity to the figure-generating revision are unconfirmed. The released StreamSampling package is outside the claim scope.

The distributional arguments assume $N \geq m \geq 1$ and strictly positive finite weights. Exact rational enumeration checks occupancy partitions and count probabilities against analytical laws. For $m = 4$, uniform-weight occupancy tests can reuse a fixed reservoir because their statistic is invariant to label order. Weighted estimator experiments

instead rebuild every reservoir. Original and repaired queries share that reservoir but use separately seeded query draws; an iid categorical control samples the full population.

For $y_i = i/N$ and $p_i = w_i/W$, define

$$\mu = \sum_i p_i y_i, \quad v = \frac{1}{m} \sum_i p_i (y_i - \mu)^2.$$

The twelve-cell extension uses $N = 10000$, $m \in \{100, 500, 1000, 2000\}$ and weights $1, i$ or $N + 1 - i$, with 5,000 fresh reservoirs per cell and disjoint seed ranges. Its primary contrast is $d_r = \hat{\mu}_{r,\text{original}} - \hat{\mu}_{r,\text{repaired}}$. Two-sided paired tests use Holm adjustment [11] across twelve contrasts. Simultaneous intervals are

$$\bar{d} \pm t_{1-0.05/(2 \cdot 12), 4999} s_d / \sqrt{5000}.$$

These Bonferroni intervals have at least 95% family coverage under their assumptions. Bias-to-reference tests form a separate 36-test family. MSE/reference variance and coverage of $\hat{\mu} \pm 1.959964\sqrt{v}$ are descriptive; this coverage uses known iid variance and a normal approximation. The locally saved protocol preceded execution, but earlier positive diagnostics motivated the grid. It is not a public preregistration. Supplement S2–S9 preserves all earlier experiments, including non-rejections and diagnostic selection.

The paired t approximation assumes independent reservoir replicates, finite variance and an adequately approximated sampling distribution of the mean difference; exact finite-sample t inference would additionally require normal differences. Post-audit diagnostics give maximum absolute sample skewness 0.083 and excess kurtosis up to 2.487 at fraction 0.01, with absolute excess kurtosis at most 0.430 otherwise. At 5,000 replicates, these shape diagnostics support the sampling approximation. Independence of pseudorandom streams remains an assumption.

We report the paired standardized difference $d_z = \bar{d}/s_d$. Detection sensitivity uses $\alpha_B = 0.05/12$ and $c = t_{1-\alpha_B/2, 4999}$. We solve $\Pr(|T_{4999, \lambda}| > c) = 0.8$ and report $\Delta_{80} = \lambda s_d / \sqrt{5000}$. The resulting threshold is a post-audit sensitivity estimate conditional on the observed SD and noncentral- t model. It was not used to plan sample size or define equivalence. The fixed Bonferroni cutoff also differs from the adaptive Holm procedure.

Results and discussion

Conversion counterexample

The unchanged conversion introduces weight categories according to the output position while advancing its next-unseen-item pointer only on one selection branch. A previously un-emitted rank can therefore be returned without advancing that pointer. A subsequent selection can reuse the same rank where a new rank was required.

For $m = 4$, $N \geq 4$, and unit weights, direct enumeration gives an excess

$$\delta_N = \frac{N - 3}{N^3}$$

in partition $(2, 2)$ and an equal deficit in $(2, 1, 1)$. Other occupancy probabilities are unchanged. One discrepant path first repeats the initial rank, then emits the second rank through the preintroduced category, then selects the remaining-mass category. The first two choices each have probability $1/N$, and the last has probability $(N - 3)/N$. In the unchanged routine the next-unseen pointer still identifies the second rank. The repaired construction advances the distinct-item state and instead produces a new rank. Since partition identities ignore rank labels, the discrepancy is not an artefact of sorting the output.

For $N = 4$, the complete exact distribution is:

Partition	IID law	Unchanged conversion	Repaired conversion
(4)	1/64	1/64	1/64
(3, 1)	3/16	3/16	3/16
(2, 2)	9/64	10/64	9/64
(2, 1, 1)	36/64	35/64	36/64
(1, 1, 1, 1)	3/32	3/32	3/32

The total variation distance on these partitions is $1/64$. The exact discrepancy establishes failure of the required occupancy law in this case. Native execution checks whether the transcription reproduces the program's behaviour.

Native execution produced 0.156210 for partition (2, 2), with Wilson [12] 95% CI [0.155500, 0.156923]. The iid occupancy law is rejected for the unchanged program ($p < 10^{-300}$), but is consistent with the exact program-specific law ($p = 0.1592$). The repaired conversion produced 0.140519, with 95% CI [0.139839, 0.141202], and its raw goodness-of-fit p-value was 0.8367. The repaired sample does not reject the reference law; its mathematical justification rests on the argument below.

At $N = 40$, $m = 4$, a post-audit diagnostic against the exact source-specific occupancy law gives $\chi_4^2 = 6.3736$ and unadjusted $p = 0.1729$. This selected, unadjusted check is descriptive; failure to reject does not establish equality of laws. The four-output failure path does not occur for $m < 4$; this statement concerns conversion, not the separate count helper.

Conversion repair

Assume an ordered reservoir contains the first m ranks of a size-biased random permutation R , so the next unseen rank has conditional probability $w_i / (W - \sum_{j=1}^k w_{R_j})$. Independent exponential keys satisfy this law in exact arithmetic. The repaired converter maintains the invariant that the distinct emitted items U_t are exactly the first $k = |U_t|$ revealed ranks. Its sampler assigns mass w_i to each seen item and the remaining mass to a residual category. A residual selection emits the next unseen rank and increments k ; a repeat leaves k unchanged.

A seen item's next-output probability is w_i / W . For an unseen item it is

$$\frac{W - \sum_{j \in U_t} w_j}{W} \frac{w_i}{W - \sum_{j \in U_t} w_j} = \frac{w_i}{W}.$$

Fresh query randomness and decisions depending only on the revealed prefix preserve the conditional rank law. Induction yields the iid product distribution. At most m ranks are exposed. The source defect breaks this invariant by activating a category before advancing the distinct-rank pointer. Nineteen small exact checks integrating rank permutations agree with the repaired product law; five original cases differ. This argument assumes correct rank retention, tie-free keys and exact arithmetic, and does not certify every floating-point execution. Supplement S4 provides the full argument.

Count-shortcut counterexample and repair

Let $P_i = \Pr(B = i)$ for $B \sim \text{Bin}(m, p)$. The recurrence is

$$\frac{P_i}{P_{i-1}} = \frac{m - i + 1}{i} \frac{p}{1 - p}.$$

The inspected shortcut instead uses a factor $m - i$ in its fast cumulative calculation. It falls back to a library quantile outside the shortcut region, so the resulting distribution cannot be inferred by simply renormalizing the wrong recurrence. Exact integration over its branching intervals is required.

At $m = 4, p = 1/4$, that integration gives:

Replacement count	Binomial law	Shortcut law
0	81/256	81/256
1	27/64	27/64
2	27/128	25/128
3	3/64	1/16
4	1/256	1/256

Probability $1/64$ is transferred from two replacements to three. The positive-truncated helper uses the same shortcut and inherits the discrepancy after conditioning. A local copy changes only the recurrence factor to $m - i + 1$; the original file remains byte-identical.

The nine-cell native diagnostic rejects three original distributions after Holm correction and none after the one-factor repair. BERN does not use this helper and serves as a control. Non-rejection is not proof of exactness; complete cells and intervals are retained in Supplement S5–S6.

Let $P_i = \binom{n}{i} p^i (1-p)^{n-i}$, $z = P_0$, $n \geq 1$, and $0 < p < 1$. The shortcut starts with $x_1 = 1 + np/(1-p) = (P_0 + P_1)/z$. With the repaired factor, step i adds

$$\frac{n(n-1) \cdots (n-i+1)p^i}{i!(1-p)^i} = P_i/z.$$

Induction gives $x_i = \sum_{j=0}^i P_j/z$, so comparing x_i to t/z performs inverse-CDF sampling; a correct fallback quantile covers the remaining interval. Restricting the uniform draw to $(z, 1)$ gives the positive-truncated law. Threshold equality has measure zero under continuous uniforms. This proof assumes exact arithmetic and a correct fallback; it does not settle finite-bit endpoint handling or floating-point errors.

Exact rational integration checks 296 distinct pairs: $n = 1, \dots, 50$, five rational probabilities, and $p = 1/n$ where distinct and defined. All repaired ordinary and truncated laws match exactly. The executable interval integration and proof are supplied separately. At $n = 1000$, 200,000 native draws per cell test five probabilities, both helper types and both versions (20 mean tests). The original ordinary and truncated means at $p = 0.004$ reject their references after Holm correction; repaired means do not. These native checks assess execution at larger counts.

Estimator effects across reservoir fractions

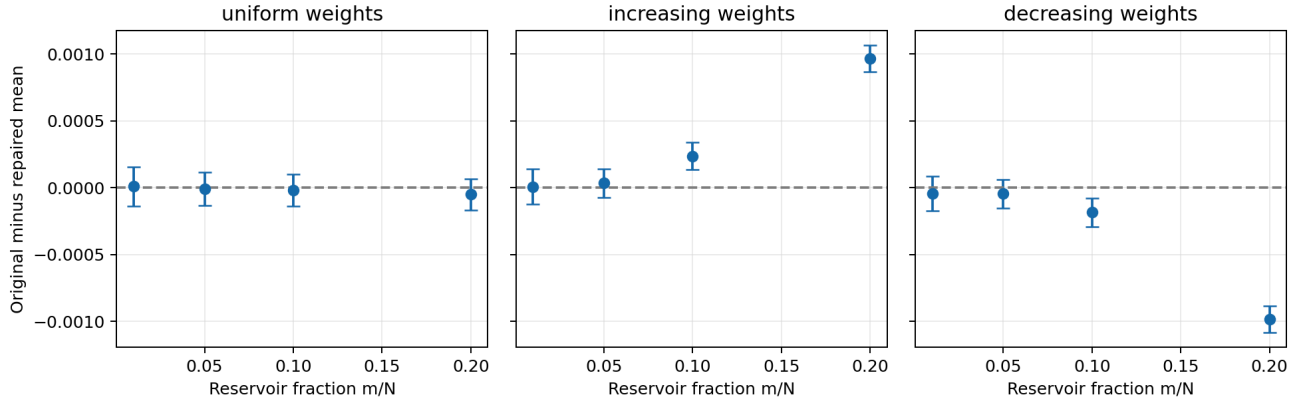


Figure 1. All twelve paired normalized-mean differences with Bonferroni family intervals. Positive values indicate a larger original estimate. Weight direction refers to association with item label, not a change in the target weight definition.

Weight profile	m/N	Original minus repaired	SD of paired difference	Simultaneous interval	Holm p	d_z	Δ_{80}
decreasing	0.01	-0.000045	0.003194	[-0.000175, 0.000084]	1	-0.014	1.67e-04
decreasing	0.05	-0.000045	0.002659	[-0.000153, 0.000063]	1	-0.017	1.39e-04
decreasing	0.1	-0.000185	0.002597	[-0.000291, -0.000080]	4.217e-06	-0.071	1.36e-04
decreasing	0.2	-0.000984	0.002503	[-0.001086, -0.000883]	$< 10^{-100}$	-0.393	1.31e-04
increasing	0.01	0.000008	0.003265	[-0.000124, 0.000141]	1	0.003	1.71e-04
increasing	0.05	0.000034	0.002638	[-0.000073, 0.000141]	1	0.013	1.38e-04
increasing	0.1	0.000236	0.002493	[0.000135, 0.000337]	2.259e-10	0.095	1.31e-04
increasing	0.2	0.000968	0.002449	[0.000869, 0.001067]	$< 10^{-100}$	0.395	1.28e-04
uniform	0.01	0.000010	0.003622	[-0.000137, 0.000157]	1	0.003	1.90e-04
uniform	0.05	-0.000009	0.003097	[-0.000134, 0.000117]	1	-0.003	1.62e-04
uniform	0.1	-0.000019	0.002975	[-0.000139, 0.000102]	1	-0.006	1.56e-04
uniform	0.2	-0.000051	0.002910	[-0.000169, 0.000067]	1	-0.017	1.53e-04

Here d_z standardizes the paired intervention difference, not the original estimator's bias. At fraction 0.2 it is approximately -0.393 and 0.395 for decreasing and increasing weights; at 0.1 it is -0.071 and 0.095 . With this many replicates, a small standardized effect can be estimated precisely. Across the grid, Δ_{80} ranges from 1.28×10^{-4} to 1.90×10^{-4} on the normalized-mean scale. This modeled detection threshold does not bound effects in cells that did not reject. Full t statistics and computed p values are retained in the analysis files; very small table values are abbreviated for readability.

The paired tests reject a zero difference for increasing and decreasing weights at fractions 0.1 and 0.2, with opposite signs. They do not reject for either profile at 0.01 or 0.05, or for uniform weights at any tested fraction. Non-rejection is not equivalence. The four-point grid is consistent with a larger effect at the largest tested fraction; it does not establish a universal monotonic law.

At fraction 0.2, original mean bias is 0.001086 for increasing weights and -0.001030 for decreasing weights, with Holm-adjusted $p < 10^{-39}$ in the separate 36-test family. None of the repaired or iid bias tests rejects. At fraction 0.1, the absolute-bias tests do not reject although the paired tests do. Pairing uses shared reservoir randomness to estimate the intervention difference more precisely; the bias tests instead compare each method with a fixed population target. The earlier 20,000-replicate follow-up also used a different sample size and test family. The complete results are reported.

The original MSE/reference-variance ratios at fraction 0.2 are 1.0346 (increasing) and 1.0618 (decreasing); corresponding nominal coverage is 0.9440 and 0.9420. Full marginal intervals are supplied in the CSV. The measured changes are modest and limited to the tested estimator and populations. The sign reversal is consistent with

erroneous rank reuse interacting with the association between weights and labels. The experiment does not determine a general bias formula. Uniform aggregate means failed to detect the separately established distributional error.

Partial replication of native performance

The preserved stage-4 design crosses $N \in \{10^6, 10^7\}$, $m/N \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$, three weight sequences, three methods and two code versions, with 20 runs per cell (2,880 records). It uses Julia 1.12.1, one thread, an Apple M3, 16 GiB RAM and macOS 27.0.1. Weight generation and construction are excluded from Add; initialization performed by Add remains included. Full-stream warmup precedes randomized method/version order within each configuration/replicate. Seeds and original records are preserved. The initial protocol undercounted the total as 1,440; the executed factorial grid was unchanged.

All 960 raw paired SKIP/BIN and SKIP/EXPJ Add ratios per code version are below one (48 method/configuration comparisons, each with 20 pairs). All 48 corresponding geometric-mean ratios per version are also below one. Thus the corrections do not create the observed ranking. However, half the 48 SKIP version/configuration cells have maximum-to-minimum time exceeding 2.5 (maximum 5.473). One BIN cell also exceeds that threshold (5.144), so high variability is not exclusive to SKIP. The observed clusters cannot be attributed to a particular hardware mechanism from these records. No timestamps, CPU affinity or power-state traces were collected in stage 4. The geometric ratios describe this run; their magnitude cannot be treated as stable across execution conditions.

Circular block resampling [13], [14] with lengths 1, 2, 4, 5 retains upper marginal sensitivity limits below one for all repaired comparisons. Twenty runs cannot establish stationarity or transferable speedup magnitudes. Repair/original intervals do not support a general claim of zero repair cost. BIN/SKIP retrieval returns stored references, whereas EXPJ sorts and converts; these interfaces must not be treated as equivalent materialized-output costs. Supplement S9 retains raw plots, timing sensitivity and measurement boundaries.

Selected reproducibility checks

Fresh Python dependencies and a new Julia depot reproduced six selected Python outputs and three native CSVs byte-for-byte on the same host. The latter contain 6.6 million output samples, including fixed-reservoir occupancy queries. The replay covers selected computations on the same host. It does not reconstruct the complete historical pipeline or test another machine. The ratio sweep ran separately in the previously validated environment. Supplement S10 records hashes, versions and the corrected launcher-path failure.

A later portable runner checks eight outputs from seven Python scripts: the six earlier outputs plus two ratio-sweep summaries. All eight were byte-identical in the recorded same-host execution. The current runner reports byte identity separately from numerical agreement: exact output files require matching bytes; decimal CSVs require matching structure, identifiers and categorical values, with explicit absolute/relative tolerances for finite numbers. Probability columns use relative tolerance only. Cross-environment byte identity has not been established. An external audit reports small decimal differences, but its execution logs were unavailable for verification.

Complexity

The conversion repair preserves the same reservoir and dynamic weighted-sampler representation. Bookkeeping adds constant work per newly emitted item and uses $\mathcal{O}(m)$ auxiliary storage. If dynamic draws and updates are expected constant-time, the conversion loop is expected $\mathcal{O}(m)$; a separate sorting step remains in the artifact and must be accounted for. A conventional comparison-sort implementation has an $\mathcal{O}(m \log m)$ sorting bound. This is distinct from counting random variates alone. The count-factor correction changes no asymptotic cost.

Exact enumeration is restricted to small cases and excludes rational-arithmetic bit costs from the following operation counts. In `exact_grid.py`, the source converter has $m!$ rank-choice leaves; count construction and sorting at each leaf give a conservative $\mathcal{O}(m \log(m) m!)$ bound, with $\mathcal{O}(m p(m) + m^2)$ storage for partition probabilities and recursive lists, where $p(m)$ is the number of integer partitions. The grid uses $m \leq 8$.

The fixed four-label `exact_audit.py` explicitly visits 4^3 converter paths and 4^4 iid reference paths. Generalizing this exhaustive labelled reference to N, m would require N^m paths, so the converter-only count is not the whole program's cost.

The weighted program `exact_weighted_conversion.py` additionally enumerates $N!$ key orders. Its conversion phase has conservative cost $\mathcal{O}(N!(N + m m!))$; building the full product-law reference and count-vector dictionaries in the implemented loops adds $\mathcal{O}(NmN^m)$. Total storage is conservatively $\mathcal{O}(NN^m + N^2 + m^2)$, including output-state dictionaries, permutation state and recursive lists. Those runs are limited to $N \leq 5, m \leq N$; the eight-draw bound of the equal-weight grid does not apply to this program. These costs concern audit enumeration, not the production reservoir algorithms.

Threats to validity

Internal validity. Rational transcriptions could differ from program semantics; native tests and controls reduce this risk. Exact-arithmetic proofs do not establish numerical safety. The separate SKIP retrieval exception for $N < m$ remains unrepaired. Extreme-weight shape checks do not verify probability laws.

Construct validity. Occupancy partitions avoid errors caused solely by sorting but discard label information. Aggregate means can miss distributional defects. Fixed-reservoir arguments apply only to the stated equal-weight invariant statistics. Weighted inference uses fresh reservoirs.

External validity. Claims concern a pinned comparison artifact. They neither disprove the formal sampling algorithms nor identify which failures affected published figures. Native timing covers synthetic streams on one device, with 20 rather than 100 repetitions; the clickstream experiment is not reproduced. The repairs are research implementations, not production certification.

Selection and inference. Diagnostic extensions followed inspection of failures. Test families are separate, with no joint 5% error-rate claim across the study. Non-rejection is not equivalence. Timing dependence, missing host-state traces and finite-bit randomness remain limitations. Local pre-execution hashes are not independently trusted timestamps.

Availability and disclosures

Original source, local repairs, independent code, raw outputs, pinned dependencies and selected replay instructions accompany the manuscript. Upstream MIT notices are preserved. The author has approved the MIT license for newly authored code; upstream and third-party notices remain applicable. Public release identifiers are pending. No upstream repository has been changed.

On 4 October 2026, the researcher reported sending a verification email to the original corresponding author. No reply has been reported; delivery and the sent message were not independently inspected. The reported contact attempt is recorded in the artifact, consistent with the venue's guidance [15]. The study remains a partial replication and audit of the comparison artifact.

AI assistance was used in code development, analysis and drafting. The author has approved this disclosure and is responsible for reviewing the final manuscript and artifact. No dedicated funding was received for this research. The author declares no conflicts of interest related to this work, confirms sole authorship and no prior collaboration with the original authors, and confirms that the manuscript is not currently submitted elsewhere.

References

- [1] A. Meligrana and A. Fazzone, “Weighted Reservoir Sampling with Replacement from Data Streams,” in *Proceedings of the ACM Web Conference 2026*, pp. 8789–8792, 2026, doi: [10.1145/3774904.3792966](https://doi.org/10.1145/3774904.3792966).
- [2] A. Meligrana and A. Fazzone, “Weighted Reservoir Sampling With Replacement from Data Streams,” arXiv:2403.20256v7, Mar. 16, 2026. [Versioned manuscript](#).

- [3] P. S. Efraimidis and P. G. Spirakis, “Weighted random sampling with a reservoir,” *Information Processing Letters*, vol. 97, no. 5, pp. 181–185, 2006, doi: [10.1016/j.ipl.2005.11.003](https://doi.org/10.1016/j.ipl.2005.11.003).
- [4] M. Shekelyan, G. Cormode, Q. Ma, A. Mohammadi Shanghooshabad, and P. Triantafillou, “Streaming Weighted Sampling over Join Queries,” in *Proceedings of EDBT 2023*, pp. 298–310, 2023, doi: [10.48786/edbt.2023.24](https://doi.org/10.48786/edbt.2023.24).
- [5] A. Meligrana, “StreamSampling.jl comparison artifact,” repository commit 408dd06f8ea9e60b412e27a73f0f277bb590cb84. [Pinned sampler source](#). Accessed: Oct. 3, 2026.
- [6] A. Bagnall, G. Stewart, and A. Banerjee, “Formally Verified Samplers from Probabilistic Programs with Loops and Conditioning,” *Proceedings of the ACM on Programming Languages*, vol. 7, PLDI, Art. 106, 2023, doi: [10.1145/3591220](https://doi.org/10.1145/3591220).
- [7] R. Bhattacharyya, S. Chakraborty, Y. Pote, U. Sarkar, and S. Sen, “Testing Self-Reducible Samplers,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 8, pp. 7952–7960, 2024, doi: [10.1609/aaai.v38i8.28632](https://doi.org/10.1609/aaai.v38i8.28632).
- [8] O. Zeyen, M. Cordy, M. Gubri, G. Perrouin, and M. Acher, “Testing Uniform Random Samplers: Methods, Datasets, and Protocols,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 10, pp. 1–24, 2026, doi: [10.1145/3797477](https://doi.org/10.1145/3797477). [Accessible preprint](#).
- [9] A. Meligrana, “StreamSampling.jl: Efficient Sampling from Data Streams in Julia,” *JuliaCon Proceedings*, vol. 8, no. 83, Art. 202, 2026, doi: [10.21105/jcon.00202](https://doi.org/10.21105/jcon.00202). [Versioned preprint consulted](#).
- [10] T. Kalibera and R. Jones, “Rigorous Benchmarking in Reasonable Time,” in *Proceedings of the 2013 International Symposium on Memory Management*, pp. 63–74, 2013, doi: [10.1145/2464157.2464160](https://doi.org/10.1145/2464157.2464160). [Corrected author manuscript and version note](#).
- [11] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Stable source](#).
- [12] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927, doi: [10.1080/01621459.1927.10502953](https://doi.org/10.1080/01621459.1927.10502953).
- [13] D. N. Politis and J. P. Romano, “A Circular Block-Resampling Procedure for Stationary Data,” Stanford University, Department of Statistics, Tech. Rep. EFS NSF 370, Mar. 1991. [Institutional record](#).
- [14] D. N. Politis and J. P. Romano, “A Circular Block-Resampling Procedure for Stationary Data,” in *Exploring the Limits of Bootstrap*, R. LePage and L. Billard, Eds. Wiley, 1992, pp. 263–270. [Author publication record](#).
- [15] ReScience C, “Overview of the submission process.” [Official guidance](#). Accessed: Oct. 3, 2026.